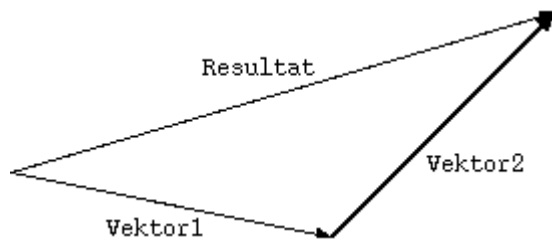


Överlagring av operatörn +

En fundamental funktion då man behandlar vektorer är möjligheten att addera vektorer. Vektorer adderas genom att placera dem efter varandra, varvid summan blir vektorn från den första vektorns startpunkt till den andra vektorns slutpunkt, enligt följande figur:

Figur 21-1. Vektoraddition



Addition av vektorerna `Vektor1` och `Vektor2` leder till resultatvektorn `Resultat`. Vill vi kunna göra denna operation med vår nuvarande definition av klassen `Vector` måste vi göra det manuellt med en speciell funktion eller metod som kan heta t.ex. `add()` eller något liknande. Vi kan istället överlagra operatörn `+`. Vi får då följande addition till klassen `Vector`:

```
// prototyp
Vector operator+ (const Vector & V) const;

// implementation
Vector Vector::operator+ (const Vector & V) const {
    Vector Result;

    // sätt den nya vektorns värden
    Result.setX ( m_X + V.m_X );
    Result.setY ( m_Y + V.m_Y );
    Result.setZ ( m_Z + V.m_Z );

    // returnera den nya vektorn
    return Result;
}
```

Prototypen skall alltså placeras headerfilen och implementationen i implemenationsfilen. Vi skall nu titta lite närmare på denna kryptiska metod. I C++ använder man nyckelordet `operator` för att berätta att man nu behandlar en operator av något slag. Man placerar den önskade operatörn efter nyckelordet `operator`, i vårt fall operatörn `+`. Detta blir alltså metodens namn. Som parameter tar metoden en annan `Vector`, som är den vektor vi vill addera till nuvarande vektor. Vi definierar `V` som en konstant referensparameter för att öka effektiviteten och eftersom vi inte vill ändra den på något sätt. Själva metoden är även `const`, eftersom vi inte ämnar ändra nuvarande vektor heller. Däremot är returtypen `Vector`. Detta är resultatet av vår metod och alltså summan av de två vektorerna. Inne i metoden `operator+` definierar vi en ny `Vector` som vi tilldelar korrekta värden på de olika koordinaterna samt till sist returnerar.

Använda överlagrade operatörer

Vi har nu definierat en överlagrad operator +, men hur används denna? Som vi redan såg så heter metoden `operator+()`. Vi kan således använda klassen på följande sätt:

```
Vector V1 ( 1, 2, 3 );
Vector V2 ( 6, 5, 4 );

// addera vektorerna
Vector Result = V1.operator+ ( V2 );
```

Inte speciellt praktiskt, eller hur? Här kommer dock en del magi in i bilden. Man kan istället för att skriva `operator+` skriva endast + och även lämna bort parentesen. Ovanstående kod kunde alltså även skrivas:

```
Vector V1 ( 1, 2, 3 );
Vector V2 ( 6, 5, 4 );

// addera vektorerna
Vector Result = V1 + V2;
```

Nu börjar det lika någonting. Vi kan nu helt transparent addera vektorer till varandra. Vi kan även addera flera än två vektorer samtidigt:

```
#include <iostream>
#include "Vector2.h"

int main () {
    Vector V1 ( 1, 2, 3 );
    Vector V2 ( 6, 5, 4 );
    Vector V3 ( -1, -2, -3 );

    // addera vektorerna
    Vector Result = V1 + V2 + V3;

    cout << "x=" << Result.x () << ", "
         << "y=" << Result.y () << ", "
         << "z=" << Result.z () << endl;
}
```

Detta fungerar eftersom den första additionen av `V1` och `V2` returnerar en ny vektor som sedan i sin tur adderas med `V3`. Vi kunde använda parenteser för att framtvunga en viss evalueringsordning, men vektoraddition är kommutativ, så ordningen spelar ingen roll. Körning av programmet ovan ger oss följande (korrekta) utskrift:

```
% ./TestVector2_2
x=6, y=5, z=4
%
```

Alla överlagrade operatörer kan anropas utan `operatorX`, där `X` är namnet på operatoren i fråga, men man kan göra det om man vill explicit visa att det är en överlagrad operator som anropas. Vi har här nått en viss transparens för vår klass `Vector`, nu återstår endel andra operatörer innan klassen är användbar i praktiken.

Överlagring av +=

Vi skall ännu i demonstrationssyfte överlagra operatoren `+=` för att skapa en metod som ändrar på objektet för vilket operatoren används. Vi kanske vill kunna skriva kod såsom t.ex.:

```
Vector V1 ( 1, 2, 3 );  
Vector V2 ( 6, 5, 4 );  
  
// addera vektor V2 till V1  
V1 += V2;
```

Det görs med följande kod:

```
// prototyp  
void operator+= (const Vector & V);  
  
// implementation  
void Vector::operator+= (const Vector & V) {  
    m_X += V.m_X;  
    m_Y += V.m_Y;  
    m_Z += V.m_Z;  
}
```

Metoden helt enkelt ändrar på medlemmarna för `v1` i exemplet ovan. Notera att metoden inte är definierad som `const` som de tidigare, eftersom den ändrar det anropande objektet.

[Företgående](#)[Överlagring av operatorer](#)[Hem](#)
[Upp](#)[Nästa](#)[Överlagring av operatörn -](#)